

Spark Torrent

Spark Torrent is a basic torrent Client which supports multi file download.

If you are already familiar with how torrent works you can directly skip to the interesting part ..

Few words about torrent

We all have used torrent some time or the other but have you thought how is it different from downloading files form a server.

In a conventional download the file is present in one server we connect to the server and the pieces of the files are sent to us one after the other as a stream.

But incase of bit torrent there is no centralized server. Rather there are many computers where the files are present known as peers. We download the file from them , not necessarily we download the entire file from only one peer.

We request some pieces from one peer and then move on to request pieces from other peer.

But know the question comes how do we find out the peers which have the file that we are trying to download.

For that we have a tracker server. It stores the hash of a file/folder with the available list of peers that have the file (complete or incomplete).

One more thing more the number of peers faster would be the download speed.

Interesting Part begins from here ...

Divided the project into the following 4 parts:

- Contents of the torrent file
- Peer List (getting the peer list from tracker)
- Downloading Blocks (downloading blocks from the peers)
- Piece/Block Handler (how blocks are being organized to write in the specific files)

Contents of the torrent file

- announce url: URL of the tracker server
- announce url list: list of urls of multiple trackers
- encoding: UTF-8 or any other type of encoding used
- piece length : Length of each piece
- Array of Files: {
 - length : total length of files in bytes
 - path: filename}

With the above information we can easily find out some other necessary information like blockSize , blockLength ,lastPieceLength , lastBlockSize.

Using putBlocksInfo to put these details directly in the torrentParser Map to be used later

Peer List

(Related files: Peer.java ,HttpTrackerUrl.java, Utils.java)

Depending on the protocol of the url (udp/http) we are requesting the tracker

In case of UDP

We have to proceed with the following step

1. Send Connection Request to tracker with a **random transaction id**
2. Receive and Parse the Connection Response to get the **announce id** and match the transaction id is same as the request and
3. Build and Send Announce Request (announce id and infoHash are the main params) required to build
4. Tracker will sent list of peers that matches with the infoHash

In case of HTTP

We only have to build a http get request containing the infoHash and other standard details and send to the announce url.

All the code related for getting the peerList is in Peers.java and HttpTrackerClient.java

Downloading Blocks

(Related files: Download.java , Utils.java)

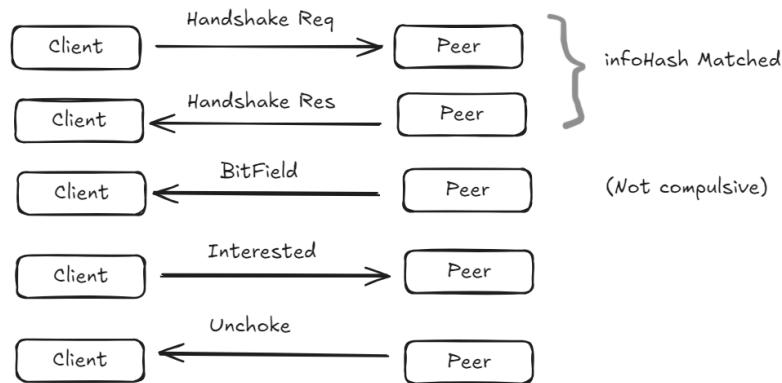
Before directly start requesting blocks from the peer we would have to do some setup with the peers.

Unlike the tracker connection which can be udp , peer connection are always TCP.

Here are we are doing the setup

All these messages are built in Message.js

- We first send a handshake message to the peer to now which files we are interested to download.
- If peer has the files will respond with handshake response .
- Next the peer can send a bitfield message to show which all pieces it has.
Or if it has decided to not share with us peer sends a choke message.
We always start as choked.
- We send a interested message if we want to download the pieces from the peer.
- After that it might unchoke message.



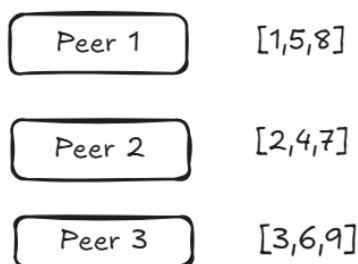
Message protocol

After a peer has unchoked us we are free to request pieces from it .
 Actually pieces are not the smallest units it is further divided into blocks.
 Each block has 16384 bytes.

We have a `pieceRequestQueue` which contains all the pieces that are to be requested.
 When all setup is done with the peer we get the first piece of the queue and send the `pieceRequest`
 During a piece Request it would continue to request all the blocks in a piece and if all the blocks are received , piece would be dequeued. And nextPiece in the queue would be requested.

All the pieces are requested asynchronously which ensures more request are made to the peers that are sending the pieces faster.

`pieceRequestQueue` = [1,2,3,4,5,6,7,8,9]



Piece/Block Handler

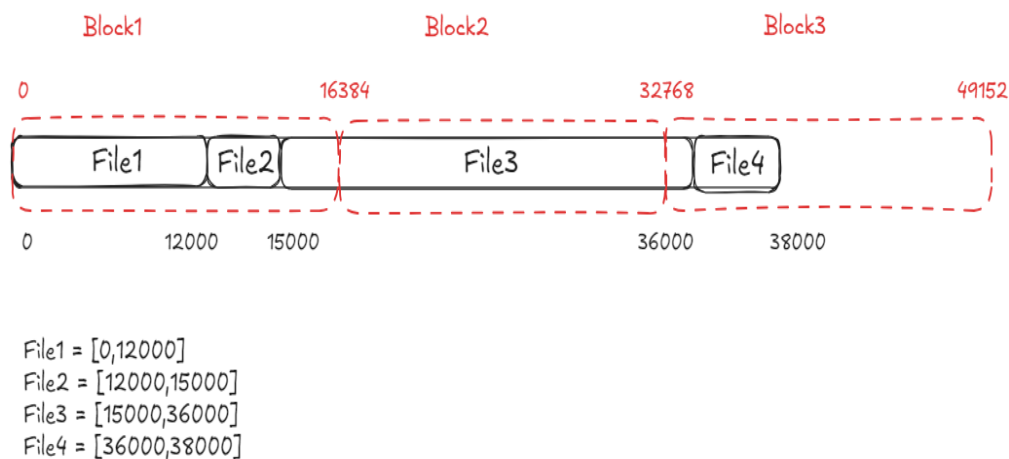
(Related files: Download.java , Utils.java)

As the block response comes from the peer we start copying the bytes from the message buffer To the file directly. This seems strait forward if we download a single file.

But when downloading multiple files , we have to decide which blocks contains data of which file
 And a single block might contain data of multiple files.

At the very first we are calculating the start and end of all the files with the filename and storing them in a data structure . I had used a list of list with 3 entries (start offset ,end offset ,filename)

All the files would come in chronological order like a stream of bytes and the order is same as Written in bit torrent file. This is how we are calculating the start and end.



This is how pieceHandler rather blockHandler algorithm works

In case of Block1

We iter through the list of list and check for which entries fall under the block 1.

In this case it would be

file1,file2 and file3

- for file1 we can just easily make a byte[] from block 0-12000 and copy it in file1 at offset 0
- But in case of file2 we have to copy the byte[] from 12000-15000 nd copy it in file2 at offset 0
- In case fo file3 copy byte[] from 15000 to 16384 to file3 at offset 0

In case of Block 2

only file3 is present

we have to copy byte[] from 16384-32768(end of block) at offset 1384 since till 1384 data is already copied from block 1

This is how it goes till now ...

Further improvements

- Better frontend
- Implement DHCP and WSS protocol to combine add more peers to the peerList